Guía de ejecución: experimentos de memoria ResNet-1D en Jetson Orin Nano con Docker

Comandos en orden, con explicación de cada uno · 27 ejecuciones (9 configuraciones × 3 repeticiones)

1. Qué se va a ejecutar

Objetivo: averiguar por qué el entrenamiento falla con `CUBLAS_STATUS_ALLOC_FAILED` cuando el contenedor tiene 2 GB, y qué factor (batch, tamaño del dataset, `empty_cache()` o simplemente el runtime) influye de verdad. Para eso se cambia **una sola variable cada vez** respecto a la configuración base y se mide la memoria en cada ejecución.

Tag	Límite de RAM	Batch	Train / Test	`empty_cache`	Para qué sirve
`probe`	2 GB	64	64 / 64	no	Un solo paso de entrenamiento. Mide la huella del runtime (Python + PyTorch + CUDA + cuBLAS) sin datos.
`base`	2 GB	64	8000 / 2000	no	Script original que falló. Referencia.
`only_bs16`	2 GB	16	8000 / 2000	no	Solo cambia el batch.
`only_data4k`	2 GB	64	4000 / 1000	no	Solo cambia el tamaño del dataset.
`only_emptycache`	2 GB	64	8000 / 2000	al inicio	Solo añade `empty_cache()`, igual que el script antiguo.
`sweep_2200m`	2,2 GB	64	8000 / 2000	no	Base con más memoria, para buscar el mínimo.
`sweep_2500m`	2,5 GB	64	8000 / 2000	no	Ídem.
`sweep_2800m`	2,8 GB	64	8000 / 2000	no	Ídem.
`sweep_3g`	3 GB	64	8000 / 2000	no	Ídem. Coincide con el antiguo Experimento 1.

Todas usan 20 épocas, 2 CPUs (`0,1`) y la misma semilla dentro de cada repetición. La repetición 1 usa la semilla 1, la 2 la semilla 2 y la 3 la semilla 3. Se repite tres veces porque un fallo de memoria cerca del límite puede no ser determinista.

2. Archivos necesarios

Solo hacen falta dos, en la misma carpeta de la Jetson:

- `entrenar_resnet1d_exp.py`: entrenamiento parametrizado y con medición de memoria.
- `resumen.py`: agrega los resultados en una tabla y un CSV.

Todos los comandos se lanzan **desde esa carpeta**, porque se monta con `-v "$PWD":/workspace`.

3. Preparación (una sola vez)

3.1 Entrar en la carpeta y crear la de resultados

```
cd /ruta/a/la/carpeta
mkdir -p results
```

`results` se monta dentro del contenedor, así que los ficheros que escribe el script sobreviven al `--rm`.

3.2 Definir la imagen Docker

```
export IMAGE=<tu_imagen_con_pytorch_para_jetson>
```

Sustituye el valor por la imagen que ya usabas en los experimentos anteriores. La variable solo vive en esta terminal: si abres otra, hay que repetir el comando.

3.3 Comprobar que la imagen ve la GPU

```
docker run --rm --runtime nvidia $IMAGE python3 -c "import torch;
print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"
```

Debe imprimir la versión de PyTorch, la de CUDA y `True`. Si sale `False`, el contenedor no está usando la GPU y todo lo que sigue no vale.

3.4 Anotar el entorno

Estos datos rellenan los huecos `[completar]` del documento del experimento:

```
cat /etc/nv_tegra_release
sudo nvpmodel -q
docker --version
stat -fc %T /sys/fs/cgroup
free -m
```

- `nv_tegra_release` da la versión de L4T/JetPack.
- `nvpmodel -q` da el modo de potencia activo.
- `stat -fc %T /sys/fs/cgroup` imprime `cgroup2fs` si el sistema usa cgroups v2 y `tmpfs` si usa v1. Importa porque cambian los ficheros de los que lee el script.
- `free -m` da la memoria total y el swap del host.

4. Anatomía de un comando

Todos los comandos tienen esta forma. Solo cambian `--memory`, `--tag`, `--seed` y los argumentos de la última línea.

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag base_r1 --seed 1 --out-dir results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/base_r1.exit
```

4.1 Opciones de Docker

Opción	Qué hace
`--rm`	Borra el contenedor al terminar. Los resultados están a salvo en `results/` del host.
`--runtime nvidia`	Da acceso a la GPU de la Jetson.
`--memory=2g`	Límite de RAM del contenedor (cgroup). Es la variable principal del experimento.
`--memory-swap=2g`	Igual que `--memory`: el contenedor no puede usar swap. Sin esto, el zram de la Jetson podría ampliar el límite real y falsear el resultado. Si el kernel no soporta swap accounting, Docker avisa y la ignora.
`--cpuset-cpus=0,1`	Restringe el contenedor a los núcleos 0 y 1.
`-v "$PWD":/workspace`	Monta la carpeta actual en `/workspace`.

Opción	Qué hace
`-w /workspace`	Directorio de trabajo dentro del contenedor.
`$IMAGE`	Imagen definida en el paso 3.2.

4.2 Argumentos del script

Argumento	Significado
`--tag`	Nombre único de la ejecución. Da nombre a los ficheros de `results/`. Termina en `_r1`, `_r2` o `_r3` según la repetición.
`--seed`	Semilla de datos e inicialización. Igual para todas las configuraciones de una misma repetición.
`--out-dir results`	Carpeta donde se escriben los resultados.
`--batch-size`	Tamaño de batch.
`--n-train` / `--n-test`	Número de muestras de entrenamiento y de test.
`--empty-cache start`	Llama a `torch.cuda.empty_cache()` al inicio, antes de usar CUDA, igual que el script antiguo.
`--max-steps 1`	Para tras un solo paso de entrenamiento. Solo se usa en `probe`.

4.3 La cola del comando

`; echo $? | tee results/TAG.exit` guarda el código de salida del contenedor. `resumen.py` lo lee después.

- `0`: terminó bien.
- `1`: el script capturó una excepción (por ejemplo `CUBLAS_STATUS_ALLOC_FAILED`) y la registró.
- `137`: el proceso murió por SIGKILL, normalmente por OOM del cgroup. No hubo tiempo de escribir el resultado final.

4.4 Qué se ve por pantalla

Cada etapa imprime una línea con la memoria en ese momento:

```
[   4.21s] cuda_init          cg=412.3 MB  anon=380.1 MB  rss=395.2 MB
cuda_alloc=0.0 MB
```

Etapas en orden: `start`, `cuda_init`, `data_ready`, `model_on_device`, `first_forward`, `first_backward`, `epoch_1` … `epoch_20`. Las cifras del ejemplo son ilustrativas, no resultados. Si falla, aparece `FALLO tras la etapa '...'` con la última etapa alcanzada. Esa etapa y la memoria en ella son el dato clave.

5. Reglas durante la ejecución

- Ejecuta los comandos **en el orden indicado** y de uno en uno. Esperan a que el anterior termine.
- Si un comando falla, **no pares**: el fallo es un resultado, no un error del experimento. Pasa al siguiente.
- Espera unos 5 segundos entre ejecuciones para que la memoria se estabilice.
- No uses la Jetson para otra cosa mientras se ejecutan (navegador, otras terminales con carga). Alteraría la memoria libre del sistema.
- Si quieres partir de caché limpia en cada ejecución, lanza antes de cada una: `sync && sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'`. Es opcional, pero conviene aplicarlo siempre o nunca, no a medias.

6. Repetición 1 (semilla 1)

6.1 `probe_r1`: huella del runtime

Un solo paso con datos mínimos. Su pico de memoria es lo que cuesta tener PyTorch, CUDA y cuBLAS inicializados.

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag probe_r1 --seed 1 --out-dir results
\
  --batch-size 64 --n-train 64 --n-test 64 --max-steps 1; echo $? | tee
results/probe_r1.exit
```

6.2 `base_r1`: configuración original

La que falló. Esperamos que falle de nuevo en `first_backward` (hipótesis), pero hay que verlo.

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag base_r1 --seed 1 --out-dir results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/base_r1.exit
```

6.3 `only_bs16_r1`: solo el batch

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_bs16_r1 --seed 1 --out-dir
results \
  --batch-size 16 --n-train 8000 --n-test 2000; echo $? | tee
results/only_bs16_r1.exit
```

6.4 `only_data4k_r1`: solo el dataset

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_data4k_r1 --seed 1 --out-dir
results \
  --batch-size 64 --n-train 4000 --n-test 1000; echo $? | tee
results/only_data4k_r1.exit
```

6.5 `only_emptycache_r1`: solo `empty_cache()`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_emptycache_r1 --seed 1 --out-
dir results \
  --batch-size 64 --n-train 8000 --n-test 2000 --empty-cache start; echo $? |
tee results/only_emptycache_r1.exit
```

6.6 `sweep_2200m_r1` : base con 2,2 GB

```
docker run --rm --runtime nvidia \
  --memory=2200m --memory-swap=2200m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2200m_r1 --seed 1 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2200m_r1.exit
```

6.7 `sweep_2500m_r1` : base con 2,5 GB

```
docker run --rm --runtime nvidia \
  --memory=2500m --memory-swap=2500m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2500m_r1 --seed 1 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2500m_r1.exit
```

6.8 `sweep_2800m_r1` : base con 2,8 GB

```
docker run --rm --runtime nvidia \
  --memory=2800m --memory-swap=2800m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2800m_r1 --seed 1 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2800m_r1.exit
```

6.9 `sweep_3g_r1` : base con 3 GB

```
docker run --rm --runtime nvidia \
  --memory=3g --memory-swap=3g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_3g_r1 --seed 1 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_3g_r1.exit
```

7. Repetición 2 (semilla 2)

Mismas nueve configuraciones que en la repetición 1, con `_r2` en el tag y `--seed 2`.

7.1 `probe_r2`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag probe_r2 --seed 2 --out-dir results
\
  --batch-size 64 --n-train 64 --n-test 64 --max-steps 1; echo $? | tee
results/probe_r2.exit
```

7.2 `base_r2`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag base_r2 --seed 2 --out-dir results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/base_r2.exit
```

7.3 `only_bs16_r2`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_bs16_r2 --seed 2 --out-dir
results \
  --batch-size 16 --n-train 8000 --n-test 2000; echo $? | tee
results/only_bs16_r2.exit
```

7.4 `only_data4k_r2`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_data4k_r2 --seed 2 --out-dir
results \
  --batch-size 64 --n-train 4000 --n-test 1000; echo $? | tee
results/only_data4k_r2.exit
```

7.5 `only_emptycache_r2`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_emptycache_r2 --seed 2 --out-
dir results \
  --batch-size 64 --n-train 8000 --n-test 2000 --empty-cache start; echo $? |
tee results/only_emptycache_r2.exit
```

7.6 `sweep_2200m_r2`

```
docker run --rm --runtime nvidia \
  --memory=2200m --memory-swap=2200m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2200m_r2 --seed 2 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2200m_r2.exit
```

7.7 `sweep_2500m_r2`

```
docker run --rm --runtime nvidia \
  --memory=2500m --memory-swap=2500m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2500m_r2 --seed 2 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2500m_r2.exit
```

7.8 `sweep_2800m_r2`

```
docker run --rm --runtime nvidia \
  --memory=2800m --memory-swap=2800m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2800m_r2 --seed 2 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2800m_r2.exit
```

7.9 `sweep_3g_r2`

```
docker run --rm --runtime nvidia \
  --memory=3g --memory-swap=3g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_3g_r2 --seed 2 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_3g_r2.exit
```

8. Repetición 3 (semilla 3)

Mismas nueve configuraciones, con `_r3` en el tag y `--seed 3`.

8.1 `probe_r3`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag probe_r3 --seed 3 --out-dir results
\
  --batch-size 64 --n-train 64 --n-test 64 --max-steps 1; echo $? | tee
results/probe_r3.exit
```

8.2 `base_r3`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag base_r3 --seed 3 --out-dir results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/base_r3.exit
```

8.3 `only_bs16_r3`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_bs16_r3 --seed 3 --out-dir
results \
  --batch-size 16 --n-train 8000 --n-test 2000; echo $? | tee
results/only_bs16_r3.exit
```

8.4 `only_data4k_r3`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_data4k_r3 --seed 3 --out-dir
results \
  --batch-size 64 --n-train 4000 --n-test 1000; echo $? | tee
results/only_data4k_r3.exit
```

8.5 `only_emptycache_r3`

```
docker run --rm --runtime nvidia \
  --memory=2g --memory-swap=2g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag only_emptycache_r3 --seed 3 --out-
dir results \
  --batch-size 64 --n-train 8000 --n-test 2000 --empty-cache start; echo $? |
tee results/only_emptycache_r3.exit
```

8.6 `sweep_2200m_r3`

```
docker run --rm --runtime nvidia \
  --memory=2200m --memory-swap=2200m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2200m_r3 --seed 3 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2200m_r3.exit
```

8.7 `sweep_2500m_r3`

```
docker run --rm --runtime nvidia \
  --memory=2500m --memory-swap=2500m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2500m_r3 --seed 3 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2500m_r3.exit
```

8.8 `sweep_2800m_r3`

```
docker run --rm --runtime nvidia \
  --memory=2800m --memory-swap=2800m --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_2800m_r3 --seed 3 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_2800m_r3.exit
```

8.9 `sweep_3g_r3`

```
docker run --rm --runtime nvidia \
  --memory=3g --memory-swap=3g --cpuset-cpus=0,1 \
  -v "$PWD":/workspace -w /workspace $IMAGE \
  python3 entrenar_resnet1d_exp.py --tag sweep_3g_r3 --seed 3 --out-dir
results \
  --batch-size 64 --n-train 8000 --n-test 2000; echo $? | tee
results/sweep_3g_r3.exit
```

9. Generar el resumen

Cuando terminen las 27 ejecuciones:

```
python3 resumen.py results
```

Escribe `results/resumen.csv` e imprime una tabla por consola con una fila por ejecución, más un recuento de éxitos por configuración. Columnas:

Columna	Significado
`status`	`ok`, `fail` (excepción capturada) o `killed` (el proceso murió sin escribir el resultado final).
`last_stage`	Última etapa alcanzada. En un fallo, indica dónde se rompió.
`exit`	Código de salida guardado con `tee`.
`val_acc`	Accuracy de validación de la última época completada.
`train_s`	Tiempo de entrenamiento en segundos.

Columna	Significado
`cg_peak_mb`	Pico de memoria total del cgroup (incluye page cache), muestreado cada 50 ms.
`cg_anon_peak_mb`	Pico de memoria anónima del cgroup. Excluye la page cache recuperable, así que es la cifra más fiel de la memoria realmente usada.
`rss_peak_mb`	Pico de RSS del proceso Python.
`cuda_max_alloc_mb`	Máximo de memoria que PyTorch asignó en la GPU. No incluye el contexto CUDA ni cuBLAS.

La columna `oom_killed` quedará vacía porque no se usa `docker inspect`. Si una ejecución sale como `killed` con `exit` 137, es un OOM del cgroup.

10. Opcional: registrar tegrastats

Para ver también el uso de RAM y GPU a nivel de sistema, abre **otra terminal** antes de empezar cada repetición:

```
sudo tegrastats --interval 200 --logfile results/tegra_r1.log
```

Párala con Ctrl+C al terminar la repetición. Usa `tegra_r2.log` y `tegra_r3.log` para las otras dos. Un único fichero por repetición mezcla las nueve configuraciones; se distinguen por la hora.

11. Cómo interpretar los resultados

Todo lo que sigue es lo que haría cada resultado más probable. Son hipótesis a contrastar con los datos, no conclusiones previas.

Observación	Lectura
`base` falla y `cg_peak_mb` ronda el límite (unos 2048 MB)	El límite del cgroup es la causa, y las reservas de GPU cuentan contra él.
`base` falla y `cg_peak_mb` está muy por debajo de 2048 MB	El cgroup no explica el fallo. Habría que investigar `nvmap` o la memoria del sistema.
`probe` ya ocupa una parte muy grande del límite	El runtime es el grueso de la memoria. Los datos y el batch importan poco.

Observación	Lectura
`base` menos `probe` ronda las decenas de MB	Coincide con la estimación teórica de datos y activaciones.
`only_bs16` pasa y `base` falla	El batch sí importa en este rango, aunque la estimación teórica no lo prevea. Habría que explicarlo.
`only_bs16` y `only_data4k` fallan igual que `base`	Ninguno de los dos factores resuelve el problema por separado.
`only_emptycache` se comporta igual que `base`	Confirma que `empty_cache()` al inicio es un no-op.
Los resultados cambian entre `_r1`, `_r2` y `_r3`	Estás en el borde del límite. Hay que repetir más veces para estimar la probabilidad de fallo.
El barrido pasa a partir de cierto valor	Ese valor es el mínimo práctico de memoria para el script base.

12. Problemas frecuentes

- **`docker: Error response ... unknown or invalid runtime name: nvidia`**: el runtime NVIDIA no está configurado en Docker. Usa el mismo método que en los experimentos anteriores para dar acceso a la GPU.
- **Aviso `Your kernel does not support swap limit capabilities`**: Docker ignora `--memory-swap`. Anótalo en el documento, porque el contenedor podría usar swap.
- **`$IMAGE` vacío (`invalid reference format`)**: has abierto otra terminal o no ejecutaste el `export` del paso 3.2.
- **`resumen.py` dice `Sin resultados`**: se ejecutó desde otra carpeta o la carpeta `results` está vacía.
- **Una ejecución sale `killed` sin nada en la columna `last_stage`**: murió antes de la primera etapa. Anótalo, no lo repitas hasta terminar el resto.

13. Qué entregar al terminar

Para rehacer las secciones de resultados del documento del experimento, hacen falta:

- `results/resumen.csv`.
- Los datos del paso 3.4.

- Opcionalmente, los `*.events.jsonl` de `probe_r1`, `base_r1` y de las ejecuciones que fallen.